`std::map` Test Suite (Extended) — Summary

> **Standard:** C++26
> **Output:** `test_map2_output.txt` (written to the working directory at run time)
> **Build:** `g++ -std=c++26 -Wall -Wextra -o test_map2 test_map2.cpp`
> *(run the executable from `src/t_map_struct_memory/` so the output file lands in the same folder)*

Conceptual Question: Would a non-minimalistic Point behave like Person?

Yes — completely.

The `struct`/`class` distinction in C++ is purely syntactic: `struct` defaults to `public` members, `class` defaults to `private`. There is no difference in memory layout, allocation behaviour, destructor mechanics, or map ownership semantics.

What determines how a type must be managed inside a `std::map` is solely whether its values are stored **by value** or **by pointer**:

Storage model	Heap allocations/entry	Map destructor cleans up?	Cleanup burden
`map<K, T>` — value	1 node (T sits inline)	Yes — full RAII	None
`map<K, T*>` — pointer	1 node + 1 T object	Node only — NOT T*	Manual `delete`

`Point` (examples 3a/3b) is stored by value → RAII.
`Point2` (examples 5a/5b) is stored as `Point2*` → requires explicit `delete`, identical to `Person*` (examples 4a/4b). The only practical difference is that `Point2` has no `std::string` member, so the number of heap allocations per entry is always exactly 2, whereas `Person*` may reach 3 or 4 when the `name_` string exceeds the SSO threshold.

File Summaries

`simple_struct.h` *(unchanged)*

Defines `Point`, a copyable plain aggregate struct with two `int32_t` members.
Stored **by value** inside map nodes; RAII-safe; trivial destructor.
See `summary.md` for the full description.

`simple_struct2.h` *(new)*

Defines `Point2`, the heap-allocated counterpart to `Point`. Same fields (`int32_t x`, `int32_t y`) and the same `to_string()` method, but with a

different intended ownership model:

- **Non-copyable** (`= delete` on copy constructor and copy-assignment operator), enforcing raw-pointer ownership semantics — the same constraint as `Person`.
- **Movable** (`= default` on move constructor and move-assignment), for symmetry with `Person`, though moves are not exercised by the examples.
- **No internal dynamic allocation**: unlike `Person`, `Point2` holds no `std::string` member. Every `map<K, Point2*>` entry therefore involves exactly **2** heap blocks (node + `Point2` object), with no conditional third allocation.
- **Destructor:** compiler-generated default — trivial, zero cost.
- **Purpose:** demonstrates that the `struct`/`class` keyword has no bearing on how a type must be owned and managed when used via raw pointer in a map.
- **Dependencies:** `<cstdint>`, `<sstream>`, `<string>`

`simple_class.h` *(unchanged)*

Defines `Person` — non-copyable, heap-allocated class.
See `summary.md` for the full description.

`map_printer.h` *(unchanged)*

Provides `display()` overloads and the `print_map<K,V>()` template.
See `summary.md` for the full description.

> **Note on `display(const Point2*)`:** this overload is **not** in `map_printer.h` (which is shared with `test_map.cpp` and knows nothing about `Point2`). It is defined at the top of `test_map2.cpp`, before any function that instantiates `print_map` with a `Point2*` value type. Two-phase ADL lookup finds it at instantiation time because `Point2` and the overload both live in the global namespace.

`test_map2.cpp` *(new)*

A copy of `test_map.cpp` extended with two additional example functions and an updated `main()`. The original eight examples are **completely unchanged**.

New additions over `test_map.cpp`:

Addition	Description
`#include "simple_struct2.h"`	pulls in `Point2`
`display(const Point2*)` overload	converts a `Point2*` to its `to_string()` representation, or `"null"`

Addition	Description
`ex5a_int_to_point2()`	`map<int32_t, Point2*>` — two heap allocations per entry, manual cleanup
`ex5b_str_to_point2()`	`map<std::string, Point2*>` — up to three heap allocations, manual cleanup
Updated `main()`	calls all 10 examples and writes to `test_map2_output.txt`

Each new example function follows the same structure as the originals: memory model block → logged insertions → map state → logged removals → map state → explicit cleanup loop.

- **Dependencies:** `<cstdint>`, `<fstream>`, `<iostream>`, `<map>`, `<string>`, `<tuple>`, `<utility>`, `"map_printer.h"`, `"simple_struct2.h"`

Overall Test Summary

The suite exercises `std::map` across **5 value types × 2 key types = 10 examples**.

Axis	Choices
Key type	`int32_t` (numeric order) · `std::string` (lexicographic order)
Value type	`int32_t` · `std::string*` · `Point` (by value) · `Person*` · `Point2*`

Examples are grouped into two memory-management categories:

RAII-safe (Examples 1 and 3): `int32_t` and `Point` values are stored inline in map nodes. The map destructor handles all cleanup automatically.

Manual ownership required (Examples 2, 4, and 5): values are raw pointers to separately heap-allocated objects. The map destructor frees nodes only. Each example shows the two-step removal pattern (erase iterator → delete pointee) and an explicit cleanup loop before scope ends.

The progression from examples 4 to 5 specifically isolates the `struct`/`class` distinction: `Person*` (class) and `Point2*` (struct) require **identical** code patterns. The only numerical difference is the maximum number of heap allocations per entry (4 for `Person*` including a possible `name_` buffer; 2 for `Point2*` which has no string member).

Individual Example Summaries

Example 1a — `std::map<int32_t, int32_t>`

- **Key/Value:** both integral, stored inline in every tree node.
- **Heap allocations per entry:** 1 (the node).

- **Cleanup:** fully automatic (RAII).
- **Insertions:** `{3,300}`, `{1,100}`, `{5,500}`, `{2,200}`, `{4,400}`.
- **Removals:** keys `5` and `2`; 3 entries remain at scope end.

Example 1b — `std::map<std::string, int32_t>`

- **Key/Value:** string key (SSO-dependent heap), `int32_t` value inline.
- **Heap allocations per entry:** 1 node + 0–1 key buffer (SSO-dependent).
- **Cleanup:** fully automatic — `~std::string()` frees key buffers.
- **Key ordering:** lexicographic.
- **Insertions:** `delta`, `alpha`, `echo`, `beta`, `gamma` with values 40/10/50/20/30.
- **Removals:** `"echo"` and `"alpha"`; 3 entries remain.

Example 2a — `std::map<int32_t, std::string*>`

- **Key/Value:** `int32_t` key inline; heap-allocated `std::string*` value.
- **Heap allocations per entry:** 2 (node + `std::string` object).
- **Cleanup:** manual. Two-step removal: erase iterator → `delete` pointer. Remaining pointers deleted in explicit loop before scope ends.
- **Insertions:** keys 10, 30, 20, 40, 50 with string values.
- **Removals:** keys `30` and `10`; remaining 3 strings deleted explicitly.

Example 2b — `std::map<std::string, std::string*>`

- **Key/Value:** `std::string` key; heap-allocated `std::string*` value.
- **Heap allocations per entry:** up to 3 (node + key buffer + value object).
- **Cleanup:** manual for value pointers; keys and nodes freed automatically.
- **Insertions:** `color`, `shape`, `weight`, `size`, `speed`.
- **Removals:** `"shape"` and `"color"`; remaining 3 strings deleted explicitly.

Example 3a — `std::map<int32_t, Point>`

- **Key/Value:** `int32_t` key and `Point` struct, both inline in the node.
- **Heap allocations per entry:** 1 (node only; `Point`'s 8 B live inside it).
- **Cleanup:** fully automatic; `~Point()` is trivial.
- **Insertions:** keys 3, 1, 5, 2, 4 with `Point` values.
- **Removals:** keys `3` and `5`; 3 entries remain at scope end.

Example 3b — `std::map<std::string, Point>`

- **Key/Value:** `std::string` key (SSO-dependent); `Point` struct inline.
- **Heap allocations per entry:** 1 node + 0–1 key buffer (SSO-dependent).
- **Cleanup:** fully automatic; `~std::string()` + trivial `~Point()`.
- **Key ordering:** lexicographic (`bot_left`, `bot_right`, `origin`, `top_left`, `top_right`).

- **Insertions:** 5 named corner/origin points.
- **Removals:** `"origin"` and `"top_right"`; 3 entries remain.

Example 4a — `std::map<int32_t, Person*>`

- **Key/Value:** `int32_t` key inline; heap-allocated `Person*` value.
- **Heap allocations per entry:** 2+ (node + `Person` object + possible `name_` string buffer when the name exceeds the SSO threshold).
- **Cleanup:** manual. Two-step removal: erase iterator → `delete Person*` (which triggers `~Person()` → `~std::string()` on `name_`). Remaining pointers deleted in explicit loop.
- **Insertions:** Alice (30), Bob (25), Charlie (35), Dave (40), Eve (28).
- **Removals:** keys `5` (Eve) and `1` (Alice); remaining 3 deleted explicitly.

Example 4b — `std::map<std::string, Person*>`

- **Key/Value:** `std::string` key (SSO-dependent); heap-allocated `Person*`.
- **Heap allocations per entry:** up to 4 (node + key buffer + `Person` object + `name_` buffer).
- **Cleanup:** manual for `Person*`; key strings and nodes freed automatically.
- **Key ordering:** lexicographic over lowercase handles.
- **Insertions:** same 5 persons as 4a, accessed via lowercase string keys.
- **Removals:** `"eve"` and `"charlie"`; remaining 3 deleted explicitly.

Example 5a — `std::map<int32_t, Point2*>` *(new)*

- **Key/Value:** `int32_t` key inline; heap-allocated `Point2*` value.
- **Heap allocations per entry:** exactly 2 — one node, one `Point2` object. Unlike `Person*`, `Point2` has no `std::string` member, so there is never a third allocation.
- **Cleanup:** manual — identical two-step pattern to examples 4a/4b. `~Point2()` is trivial (no resources to release), so `delete p` frees the heap block without any chain of destructor calls.
- **struct vs. class:** the use of `struct` instead of `class` makes no difference to the map code. Both `Point2` and `Person` are non-copyable types stored via raw pointer; their insertion, removal, and cleanup code is structurally identical.
- **Insertions:** keys 3, 1, 5, 2, 4 with `Point2` coordinate values.
- **Removals:** keys `5` and `1`; remaining 3 `Point2*` deleted explicitly.

Example 5b — `std::map<std::string, Point2*>` *(new)*

- **Key/Value:** `std::string` key (SSO-dependent); heap-allocated `Point2*`.

- **Heap allocations per entry:** up to 3 (node + key buffer + `Point2` object). The short keys used (`"origin"` etc.) fall within the SSO range on most implementations, so in practice 2 allocations are expected.
- **Cleanup:** manual for `Point2*`; key strings and nodes freed automatically. Same pattern as example 4b.
- **Key ordering:** lexicographic (`bot_left`, `bot_right`, `origin`, `top_left`, `top_right`).
- **Insertions:** 5 named corner/origin points as `Point2*` objects.
- **Removals:** `"origin"` and `"top_right"`; remaining 3 `Point2*` deleted explicitly.
- **Key takeaway:** the map code for `map<std::string, Point2*>` and `map<std::string, Person*>` (example 4b) is written and cleaned up in exactly the same way. Only the type name and the absence of a string member in `Point2` differ.